

NULL Handling — Semantics, COALESCE, NULLIF, and Three-Valued Logic

Chapter 14 | *PostgreSQL Complete Guide* Module: 02_SQL_Basics | Difficulty: Beginner–Intermediate |

Estimated Reading Time: 35 minutes

Table of Contents

- [Learning Objectives](#)
 - [Theory: What is NULL?](#)
 - [Three-Valued Logic](#)
 - [IS NULL and IS NOT NULL](#)
 - [NULL in Comparisons](#)
 - [COALESCE](#)
 - [NULLIF](#)
 - [NVL-Style Patterns](#)
 - [NULL in Aggregations](#)
 - [NULL in JOINS](#)
 - [ASCII Visual Diagrams](#)
 - [SQL Examples](#)
 - [Common Mistakes](#)
 - [Best Practices](#)
 - [Performance Considerations](#)
 - [Interview Questions](#)
 - [Interview Answers](#)
 - [Hands-on Exercises](#)
 - [Solutions](#)
 - [Advanced Notes](#)
 - [Cross-References](#)
-

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what NULL represents and why it exists in relational databases
 - Apply three-valued logic (TRUE/FALSE/UNKNOWN) to correctly reason about NULL in conditions
 - Use IS NULL, IS NOT NULL, IS DISTINCT FROM for NULL-safe comparisons
 - Replace NULLs with defaults using COALESCE and convert values to NULL with NULLIF
 - Understand how NULL propagates through aggregations, joins, and expressions
-

Theory: What is NULL?

NULL is not a value — it is a **marker for the absence of a value**. It represents "unknown", "not applicable", or "missing". NULL is distinct from zero, empty string, or false.

Why NULL Exists

In the real world, data is often incomplete:

- An employee's manager is unknown (new employee, no manager yet) — `manager_id IS NULL`
- A product has no discount — `discount IS NULL` (not the same as `discount = 0`)
- A phone number was not provided — `phone IS NULL`

E.F. Codd introduced NULL into the relational model specifically to handle incomplete information without using sentinel values (like -1, 'N/A', or 0).

NULL is Not Equal to NULL

This is the most important rule: **NULL = NULL evaluates to UNKNOWN, not TRUE.**

```
SELECT NULL = NULL;      -- returns NULL (UNKNOWN)
SELECT NULL IS NULL;     -- returns TRUE
SELECT NULL IS NOT NULL; -- returns FALSE
```

NULL Propagation

Any arithmetic or string operation involving NULL produces NULL:

```
SELECT 5 + NULL;      -- NULL
SELECT 'hello' || NULL; -- NULL
SELECT UPPER(NULL);   -- NULL
SELECT NULL * 100;    -- NULL
```

Three-Valued Logic

Standard Boolean logic has two values: TRUE and FALSE. SQL has three: **TRUE, FALSE, and UNKNOWN** (which results from NULL comparisons).

Truth Tables

AND:

TRUE	AND	TRUE	=	TRUE
TRUE	AND	FALSE	=	FALSE
TRUE	AND	UNKNOWN	=	UNKNOWN
FALSE	AND	FALSE	=	FALSE
FALSE	AND	UNKNOWN	=	FALSE
UNKNOWN	AND	UNKNOWN	=	UNKNOWN

<-- FALSE dominates in AND

OR:

TRUE	OR	TRUE	=	TRUE
TRUE	OR	FALSE	=	TRUE
TRUE	OR	UNKNOWN	=	TRUE
FALSE	OR	FALSE	=	FALSE
FALSE	OR	UNKNOWN	=	UNKNOWN
UNKNOWN	OR	UNKNOWN	=	UNKNOWN

<-- TRUE dominates in OR

NOT:

NOT	TRUE	=	FALSE
-----	------	---	-------

```
NOT FALSE = TRUE
NOT UNKNOWN = UNKNOWN      <-- NOT does NOT resolve NULL
```

WHERE Clause and UNKNOWN

The WHERE clause only passes rows where the expression is **TRUE**. Rows evaluating to FALSE or UNKNOWN (NULL) are both excluded.

```
-- Assume employee has commission_pct = NULL
-- This row is EXCLUDED from both queries:
SELECT * FROM employees WHERE commission_pct > 0;    -- NULL > 0 = UNKNOWN
SELECT * FROM employees WHERE commission_pct <= 0;    -- NULL <= 0 = UNKNOWN
-- The row appears in NEITHER result set!
```

IS NULL and IS NOT NULL

The correct way to test for NULL:

```
-- Find employees with no manager
SELECT * FROM employees WHERE manager_id IS NULL;

-- Find employees who have a manager
SELECT * FROM employees WHERE manager_id IS NOT NULL;

-- WRONG: These never match any row
SELECT * FROM employees WHERE manager_id = NULL;    -- always UNKNOWN
SELECT * FROM employees WHERE manager_id != NULL;    -- always UNKNOWN
```

IS DISTINCT FROM (NULL-safe equality)

```
-- IS DISTINCT FROM: treats NULL as equal to NULL
SELECT NULL IS DISTINCT FROM NULL;    -- FALSE (they are the same)
SELECT 1 IS DISTINCT FROM NULL;    -- TRUE (different)
SELECT 1 IS DISTINCT FROM 1;    -- FALSE (same)

-- Practical: find rows where old_price differs from new_price (handling NULLs)
SELECT * FROM price_history
WHERE old_price IS DISTINCT FROM new_price;
-- This correctly handles: NULL vs NULL (same), NULL vs 5 (different), 5 vs 5 (same)
```

COALESCE

COALESCE(val1, val2, ..., valN) returns the **first non-NULL argument**. If all arguments are NULL, returns NULL.

Basic Usage

```
-- Return discount; if NULL, return 0
SELECT product_name,
       COALESCE(discount, 0) AS effective_discount
FROM   products;

-- Fallback chain
SELECT COALESCE(preferred_name, first_name, 'Unknown') AS display_name
FROM   employees;
```

COALESCE in Calculations

```
-- Prevent NULL propagation in arithmetic
SELECT salary + COALESCE(commission, 0) AS total_comp
FROM   employees;

-- Without COALESCE: salary + NULL = NULL (incorrect)
```

COALESCE for Empty String to NULL

```
-- Treat empty string as NULL
SELECT COALESCE(NULLIF(phone, ''), 'No phone') AS phone_display
FROM   contacts;
```

Short-Circuit Evaluation

COALESCE short-circuits — once a non-NULL value is found, remaining arguments are not evaluated (important for expensive function calls):

```
-- expensive_function() is only called if cheap_lookup() returns NULL
SELECT COALESCE(cheap_lookup(id), expensive_function(id)) FROM t;
```

NULLIF

`NULLIF(val1, val2)` returns NULL if `val1 = val2`, otherwise returns `val1`. It is the inverse of COALESCE.

Basic Usage

```
-- Convert 0 to NULL (avoid division by zero)
SELECT total_sales / NULLIF(total_orders, 0) AS avg_order_value
FROM   monthly_stats;

-- Convert empty string to NULL
SELECT NULLIF(middle_name, '') AS middle_name
FROM   contacts;

-- Convert sentinel values to NULL
```

```
SELECT NULLIF(error_code, -1) AS real_error_code
FROM   system_logs;
```

NULLIF for Safe Division

```
-- Without NULLIF: division by zero raises ERROR
SELECT 100 / 0; -- ERROR: division by zero

-- With NULLIF: gracefully returns NULL
SELECT 100 / NULLIF(0, 0); -- returns NULL
```

NVL-Style Patterns

Oracle's NVL function is equivalent to COALESCE with two arguments. PostgreSQL does not have NVL, but COALESCE is a direct replacement:

```
-- Oracle:      NVL(col, default_val)
-- PostgreSQL:  COALESCE(col, default_val)

-- Oracle:      NVL2(col, if_not_null, if_null)
-- PostgreSQL:  CASE WHEN col IS NOT NULL THEN if_not_null ELSE if_null END
-- Or:          COALESCE substituted via IIF-equivalent

-- Simulate NVL2:
SELECT employee_id,
       CASE WHEN commission_pct IS NOT NULL
            THEN salary + salary * commission_pct
            ELSE salary
       END AS total_comp
FROM   employees;
```

CASE WHEN for Complex NULL Handling

```
SELECT product_id,
       CASE
           WHEN price IS NULL      THEN 'Price not set'
           WHEN price = 0          THEN 'Free'
           WHEN price < 10         THEN 'Budget'
           ELSE                    'Standard'
       END AS price_tier
FROM   products;
```

NULL in Aggregations

Aggregate functions ignore NULL values (except COUNT(*)):

Function	NULL Behavior
----------	---------------

COUNT(*)	Counts ALL rows including those with NULLs
COUNT(col)	Counts only non-NULL values in col
SUM(col)	Ignores NULLs; returns NULL if ALL values are NULL
AVG(col)	Ignores NULLs in both numerator and denominator
MIN(col)	Ignores NULLs
MAX(col)	Ignores NULLs

```
-- 10 employees, 3 have commission_pct = NULL
SELECT COUNT(*)          AS total_rows,          -- 10
       COUNT(commission_pct) AS rows_with_commission, -- 7
       SUM(commission_pct)   AS total_commission,   -- sum of 7 values
       AVG(commission_pct)   AS avg_commission      -- avg of 7 values (not 10)
FROM   employees;
```

Treating NULL as Zero in Aggregations

```
-- If you want NULLs counted as 0 in the average:
SELECT AVG(COALESCE(commission_pct, 0)) AS avg_with_nulls_as_zero
FROM   employees;
```

NULL in JOINS

NULLs in join keys cause rows to be excluded from INNER JOINS and to appear only in the "outer" side of OUTER JOINS.

NULL Join Key

```
-- employees table has some rows with department_id = NULL
-- INNER JOIN: rows with NULL department_id are excluded
SELECT e.first_name, d.department_name
FROM   employees e
INNER JOIN departments d ON e.department_id = d.department_id;
-- Employees with department_id IS NULL do NOT appear

-- LEFT JOIN: rows with NULL department_id appear with NULL department columns
SELECT e.first_name, d.department_name
FROM   employees e
LEFT JOIN departments d ON e.department_id = d.department_id;
-- Employees with NULL department_id appear with department_name = NULL
```

Anti-Join Pattern Using NULL in LEFT JOIN

```
-- Find employees who are NOT in any department (NULL join key trap avoided)
SELECT e.employee_id, e.first_name
FROM   employees e
LEFT JOIN departments d ON e.department_id = d.department_id
WHERE  d.department_id IS NULL; -- rows that did not join
```

ASCII Visual Diagrams

Three-Valued Logic Matrix

AND		TRUE		FALSE		UNKNOWN
-----+-----+-----+-----						
TRUE		TRUE		FALSE		UNKNOWN
FALSE		FALSE		FALSE		FALSE
UNKN		UNKNOWN		FALSE		UNKNOWN
OR		TRUE		FALSE		UNKNOWN
-----+-----+-----+-----						
TRUE		TRUE		TRUE		TRUE
FALSE		TRUE		FALSE		UNKNOWN
UNKN		TRUE		UNKNOWN		UNKNOWN
NOT		Result				
-----+-----						
TRUE		FALSE				
FALSE		TRUE				
UNKN		UNKNOWN				

COALESCE Evaluation Flow

```
COALESCE(a, b, c, d)

a IS NOT NULL?
|
+-- YES --> return a
|
+-- NO --> b IS NOT NULL?
            |
            +-- YES --> return b
            |
            +-- NO --> c IS NOT NULL?
                    |
                    +-- YES --> return c
                    |
                    +-- NO --> d IS NOT NULL?
                            |
                            +-- YES --> return d
                            |
                            +-- NO --> return NULL
```

NULL in JOIN

employees				departments	
id	name	dept_id		dept_id	dept_name
1	Alice	10		10	HR
2	Bob	20		20	IT
3	Carol	NULL			

INNER JOIN result (Carol excluded):

id	name	dept_name
1	Alice	HR
2	Bob	IT

LEFT JOIN result (Carol included with NULL dept_name):

id	name	dept_name
1	Alice	HR
2	Bob	IT
3	Carol	NULL

SQL Examples

```
-- Example 1: IS NULL
SELECT * FROM employees WHERE manager_id IS NULL;

-- Example 2: IS NOT NULL
SELECT * FROM orders WHERE shipped_date IS NOT NULL;

-- Example 3: NULL in arithmetic
SELECT employee_id, salary, commission_pct,
       salary + salary * commission_pct AS total_wrong,    -- NULL if commission_pct
IS NULL
       salary + salary * COALESCE(commission_pct, 0) AS total_correct
FROM employees;

-- Example 4: COALESCE with default
SELECT product_name, COALESCE(description, 'No description available') AS
description
FROM products;

-- Example 5: COALESCE fallback chain
```



```
SELECT COALESCE(nickname, preferred_name, first_name, 'Unknown') AS display_name
FROM users;
```

-- Example 6: NULLIF to avoid division by zero

```
SELECT category,
       total_revenue,
       total_orders,
       total_revenue / NULLIF(total_orders, 0) AS avg_order_value
FROM category_stats;
```

-- Example 7: NULLIF to convert sentinel to NULL

```
SELECT NULLIF(error_code, 0) AS meaningful_error
FROM transactions;
```

-- Example 8: IS DISTINCT FROM NULL-safe comparison

```
SELECT a.product_id
FROM products_v1 a
JOIN products_v2 b ON a.product_id = b.product_id
WHERE a.price IS DISTINCT FROM b.price;
```

-- Example 9: COUNT(*) vs COUNT(col)

```
SELECT COUNT(*)          AS total_employees,
       COUNT(commission_pct) AS employees_with_commission,
       COUNT(*) - COUNT(commission_pct) AS employees_without_commission
FROM employees;
```

-- Example 10: SUM/AVG ignore NULLs

```
SELECT SUM(commission_pct) AS sum_ignores_nulls,
       AVG(commission_pct) AS avg_ignores_nulls,
       AVG(COALESCE(commission_pct, 0)) AS avg_treats_null_as_zero
FROM employees;
```

-- Example 11: NULL in ORDER BY (NULLs last)

```
SELECT employee_id, commission_pct
FROM employees
ORDER BY commission_pct NULLS LAST;
```

-- Example 12: NULL in GROUP BY (NULLs grouped together)

```
SELECT department_id, COUNT(*) AS headcount
FROM employees
GROUP BY department_id; -- NULL department_id forms its own group
```

-- Example 13: LEFT JOIN to find unmatched rows (anti-join)

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL; -- customers with no orders
```

-- Example 14: NVL2 simulation

```
SELECT employee_id,
       CASE WHEN commission_pct IS NOT NULL
            THEN 'Commissioned'
```

```

        ELSE 'Salaried only'
    END AS compensation_type
FROM employees;

-- Example 15: COALESCE in UPDATE
UPDATE employees
SET phone = COALESCE(phone, 'TBD');

-- Example 16: Combining NULLIF and COALESCE
SELECT COALESCE(NULLIF(TRIM(address), ''), 'Address not provided') AS clean_address
FROM contacts;

-- Example 17: Handling NULL in string concatenation
SELECT first_name || ' ' || COALESCE(middle_name || ' ', '') || last_name AS
full_name
FROM employees;

-- Example 18: NULL-safe unique constraint consideration
-- Two NULLs in a UNIQUE column are allowed in PostgreSQL (NULLs are distinct)
INSERT INTO users (email) VALUES (NULL); -- OK
INSERT INTO users (email) VALUES (NULL); -- Also OK -- two NULLs allowed in UNIQUE
column

-- Example 19: FILTER clause to count NULLs conditionally
SELECT COUNT(*) FILTER (WHERE commission_pct IS NULL) AS null_commissions,
       COUNT(*) FILTER (WHERE commission_pct IS NOT NULL) AS non_null_commissions
FROM employees;

-- Example 20: Propagation in CASE
SELECT CASE WHEN NULL THEN 'yes' ELSE 'no' END; -- returns 'no' (NULL is not TRUE)

```

Common Mistakes

1. **Using `= NULL` or `!= NULL`**. These always evaluate to UNKNOWN. Always use `IS NULL` or `IS NOT NULL`.
2. **Assuming `COUNT(col)` counts all rows**. `COUNT(col)` skips NULLs. Use `COUNT(*)` for total row count.
3. **`NULL + anything = NULL`**. Forgetting to wrap nullable columns in `COALESCE` before arithmetic operations results in silently wrong calculations that appear as NULL.
4. **`NOT IN` with NULLs**. `col NOT IN (1, NULL, 3)` returns UNKNOWN for every row. Always filter NULLs from NOT IN lists or use NOT EXISTS.
5. **NULLs excluded from INNER JOINS**. If a join key is NULL in one table, that row will never match and will not appear in an INNER JOIN result. Use LEFT JOIN and check for NULLs explicitly.
6. **AVG including NULL rows in the denominator**. AVG ignores NULLs in both numerator and count — `AVG(col)` over 10 rows where 3 are NULL divides by 7, not 10. Use `AVG(COALESCE(col, 0))` if you want all 10 in the denominator.

7. **Assuming two NULLs are equal in UNIQUE constraints.** In PostgreSQL, a UNIQUE constraint allows multiple NULL values because `NULL ≠ NULL`. If you want at most one NULL, use a partial unique index: `CREATE UNIQUE INDEX ... WHERE col IS NOT NULL`.
-

Best Practices

1. **Design columns as NOT NULL where possible** — forces data quality at the schema level. Use DEFAULT values to avoid NULL where semantically appropriate.
 2. **Use COALESCE defensively in arithmetic** — wrap every nullable column in COALESCE before using it in calculations or string concatenations.
 3. **Use IS DISTINCT FROM for nullable column comparisons** in UPDATE/MERGE to detect actual changes without NULL false-positives.
 4. **Document NULL semantics** — in comments or column descriptions, state what NULL means for each nullable column (is it "unknown", "not applicable", or "not yet set"?).
 5. **Use NOT EXISTS instead of NOT IN** when the exclusion set comes from a subquery or could contain NULLs.
 6. **Apply NULLS FIRST / NULLS LAST explicitly** when NULLs must appear at a specific position in ORDER BY, rather than relying on database defaults.
 7. **Use the FILTER clause in aggregations** (`COUNT(*) FILTER (WHERE ...)`) instead of CASE WHEN tricks — it is cleaner and more readable.
-

Performance Considerations

Index Behavior with NULL

Standard B-tree indexes in PostgreSQL **do** index NULL values, unlike some other databases. This means `WHERE col IS NULL` can use a B-tree index.

```
-- This CAN use an index on col
SELECT * FROM large_table WHERE nullable_col IS NULL;
```

Partial Indexes to Exclude NULLs

```
-- Index only the non-NULL rows (smaller index, faster on non-NULL queries)
CREATE INDEX idx_employees_commission
ON employees (commission_pct)
WHERE commission_pct IS NOT NULL;
```

COALESCE and Index Use

Wrapping a column in COALESCE in a WHERE clause can prevent index use:

```
-- May NOT use index on commission_pct:
WHERE COALESCE(commission_pct, 0) > 0.1

-- Better – use OR to keep index eligibility:
WHERE commission_pct > 0.1 -- indexes can be used here
-- (NULL rows are excluded anyway by > comparison)
```

Interview Questions

1. What does NULL represent in SQL, and how is it different from zero or empty string?
 2. What is three-valued logic, and why does it matter for WHERE clauses?
 3. Why does `WHERE col = NULL` never return rows?
 4. What is the difference between `COUNT(*)` and `COUNT(col)`?
 5. How does `COALESCE` work, and how does it differ from `NULLIF`?
 6. Explain a scenario where `NOT IN` fails because of NULLs, and provide the fix.
 7. How do NULLs behave in an `INNER JOIN` vs a `LEFT JOIN`?
 8. What does `NULL IS DISTINCT FROM NULL` evaluate to?
 9. Can a `UNIQUE` constraint column have multiple NULL values in PostgreSQL?
 10. What is the result of `NOT UNKNOWN` in three-valued logic?
-

Interview Answers

Q1: What is NULL? NULL represents the absence of a value — "unknown" or "not applicable". It is not zero, not an empty string, and not false. It is a special marker in the relational model introduced to handle missing or inapplicable data without using sentinel values.

Q2: Three-valued logic? SQL Boolean expressions can evaluate to TRUE, FALSE, or UNKNOWN. UNKNOWN arises from any comparison involving NULL (e.g., `NULL > 5` is UNKNOWN). WHERE passes only TRUE rows; both FALSE and UNKNOWN rows are excluded. This is critical for understanding why some rows "disappear" from queries.

Q3: WHERE col = NULL never returns rows? `col = NULL` evaluates to UNKNOWN (not TRUE, not FALSE) for every row, including rows where col actually IS NULL. The WHERE clause requires TRUE, so UNKNOWN excludes all rows. Use `IS NULL` instead.

Q4: COUNT(*) vs COUNT(col)? `COUNT(*)` counts all rows in the group regardless of NULLs. `COUNT(col)` counts only rows where col is NOT NULL. If col has 5 NULLs out of 100 rows, `COUNT(*) = 100` and `COUNT(col) = 95`.

Q5: COALESCE vs NULLIF? `COALESCE` returns the first non-NULL argument — used to replace NULL with a default value. `NULLIF` returns NULL when two arguments are equal — used to convert a specific value to NULL. They are often combined: `COALESCE(NULLIF(col, ''), 'default')` converts empty string to NULL then replaces NULL with 'default'.

Q6: NOT IN with NULLs? `dept_id NOT IN (10, NULL, 30)` evaluates as `dept_id <> 10 AND dept_id <> NULL AND dept_id <> 30`. The middle term is always UNKNOWN, making the whole AND expression UNKNOWN for every row. Fix: use `NOT EXISTS` or filter NULLs from the list.

Q7: NULL in JOINS? In an `INNER JOIN`, rows with NULL in the join key are excluded (`NULL = NULL` is UNKNOWN, not TRUE, so no match occurs). In a `LEFT JOIN`, the left-side row is preserved with NULL-filled right-side columns when there is no match — including when the key is NULL.

Q8: NULL IS DISTINCT FROM NULL ? FALSE. IS DISTINCT FROM is NULL-safe: it treats two NULLs as equal. Therefore, NULL is NOT distinct from NULL.

Q9: Multiple NULLs in UNIQUE column? Yes, in PostgreSQL (and standard SQL), a UNIQUE constraint does not prevent multiple NULL values because NULLs are considered distinct from each other. To enforce at-most-one-NULL, create a partial unique index: `CREATE UNIQUE INDEX ON t (col) WHERE col IS NOT NULL`.

Q10: NOT UNKNOWN? UNKNOWN. Applying NOT to UNKNOWN still yields UNKNOWN. This means `NOT (col = NULL)` does not match rows where col IS NULL — it evaluates to NOT UNKNOWN = UNKNOWN, which is still excluded from WHERE.

Hands-on Exercises

Setup:

```
CREATE TABLE employees_demo (  
    emp_id          SERIAL PRIMARY KEY,  
    first_name      TEXT NOT NULL,  
    last_name       TEXT NOT NULL,  
    salary          NUMERIC(10,2) NOT NULL,  
    commission      NUMERIC(5,4),    -- nullable  
    manager_id      INT,              -- nullable  
    department      TEXT              -- nullable  
);  
  
INSERT INTO employees_demo (first_name, last_name, salary, commission, manager_id,  
department) VALUES  
( 'Alice', 'Smith', 70000, 0.10, 1, 'Sales'),  
( 'Bob', 'Jones', 60000, NULL, 1, 'Sales'),  
( 'Carol', 'Williams', 80000, 0.15, NULL, 'Management'),  
( 'David', 'Brown', 55000, NULL, 3, NULL),  
( 'Eve', 'Taylor', 65000, 0.05, 3, 'IT'),  
( 'Frank', 'Wilson', 72000, NULL, NULL, 'IT'),  
( 'Grace', 'Davis', 58000, 0.08, 1, NULL);
```

Exercise 1: Calculate total compensation (salary + salary * commission) for all employees. Show 0 for commission where NULL.

Exercise 2: Find employees who either have no manager OR have no department (use IS NULL).

Exercise 3: Count: total employees, employees with commission, employees without commission.

Exercise 4: Use COALESCE to produce a "contact label": if department is set, show it; otherwise show 'Unassigned'.

Exercise 5: Find employees where salary IS DISTINCT FROM 70000 (this should also match NULLs if salary were nullable — demonstrate the concept with commission instead: find rows where commission IS DISTINCT FROM 0.10).

Solutions

```

-- Exercise 1
SELECT first_name, last_name, salary,
       COALESCE(commission, 0) AS commission,
       salary + salary * COALESCE(commission, 0) AS total_comp
FROM   employees_demo;

-- Exercise 2
SELECT first_name, last_name, manager_id, department
FROM   employees_demo
WHERE  manager_id IS NULL OR department IS NULL;

-- Exercise 3
SELECT COUNT(*)           AS total_employees,
       COUNT(commission) AS with_commission,
       COUNT(*) - COUNT(commission) AS without_commission
FROM   employees_demo;

-- Exercise 4
SELECT first_name, last_name,
       COALESCE(department, 'Unassigned') AS dept_label
FROM   employees_demo;

-- Exercise 5
SELECT first_name, last_name, commission
FROM   employees_demo
WHERE  commission IS DISTINCT FROM 0.10;
-- Returns all rows where commission != 0.10 OR commission IS NULL
-- Standard != would miss NULL rows

```

Advanced Notes

NULL in Window Functions

Window functions also ignore NULLs in some contexts. `FIRST_VALUE`, `LAST_VALUE`, and `NTH_VALUE` have `IGNORE NULLS` / `RESPECT NULLS` options (PostgreSQL 16+):

```

-- Skip NULLs when finding the first non-null value in a window
SELECT emp_id, salary,
       FIRST_VALUE(commission) IGNORE NULLS OVER (ORDER BY emp_id)
FROM   employees_demo;

```

NULL and Text Search

`tsvector` and `tsquery` operations treat NULL as absent. Concatenating NULL into a `tsvector` returns NULL; wrap with `COALESCE`:

```

SELECT to_tsvector('english', COALESCE(description, '')) FROM products;

```

NULL in Arrays

PostgreSQL arrays can contain NULL elements:

```
SELECT ARRAY[1, NULL, 3];      -- {1,NULL,3}
SELECT ARRAY[1, NULL, 3][2];  -- NULL
SELECT 1 = ANY(ARRAY[1, NULL, 3]); -- TRUE (found a match before NULL)
SELECT 4 = ANY(ARRAY[1, NULL, 3]); -- NULL (couldn't confirm absence)
```

pg_catalog and NULL Semantics

The catalog uses NULLs extensively (e.g., `pg_attribute.attdefaultval` is NULL when there's no default). Understanding NULL semantics is essential when querying system catalogs.

Cross-References

- **Previous:** [04 filtering and operators.md](#) — NOT IN NULL trap, IS DISTINCT FROM
- **Next:** [06 string functions.md](#) — String functions that handle NULL
- **Related:** [03 select basics.md](#) — NULL ordering in ORDER BY
- **Related (03_Intermediate):** [03_Intermediate_SQL/02_aggregations.md](#) — NULL behavior in GROUP BY and HAVING
- **Related (03_Intermediate):** [03_Intermediate_SQL/01_joins.md](#) — NULL behavior in INNER vs OUTER joins